Lecture 9: Synthetic Control Methods: Impacts of Optimization Windows and Variable Scale

James Sears*
AFRE 891/991 FS 26
Michigan State University

Table of Contents

1. **Optimization Window**
2. **Impacts of Scale**

Prologue

Packages we'll use today:

```
if (!require("pacman")) install.packages("pacman")
pacman::p_load(fixest,  tidysynth, tidyverse)
```

Let's load in the German reunification data as well:

```
deu ← haven::read_dta("data/repgermany.dta") %>%
    mutate_at(vars(index, year, gdp, infrate, trade, schooling,
                   invest60, invest70, invest80,
                   industry),
              as.numeric) %>%
    mutate_at(vars(country), as.character)
```

German Reunification

Let's set up our initial synthetic control to replicate **Abadie, Diamond, and Hainmueller (2010)**

```
synth ← deu %>%
    synthetic_control(
      outcome = gdp, # outcome
      unit = country, # unit identifier
      time = year, # time identifier
      i_unit = "West Germany", # treated unit
      i_time = 1990, # treatment year
      generate_placebos = T # whether to generate placebos for inference
    ) %>%
  generate_predictor(time_window = 1981:1990,
                     gdp_81_90 = mean(gdp, na.rm = T),
                     trade_81_90 = mean(trade, na.rm = T),
                     infrate_81_90 = mean(infrate, na.rm = T)
  ) %>%
  generate_predictor(time_window = 1971:1980,
                     industry_71_80 = mean(industry, na.rm = T)) %>%
  generate_predictor(time_window =c(1970, 1975),
                     schooling_70_75 = mean(schooling, na.rm = T)) %>%
  generate_predictor(time_window = 1980,invest_80 = invest80)
```

Optimization Window

Optimization Window

What is the `optimization_window` argument of `generate_weights()` doing if we manually set the time for our predictors?

A: tidysynth follows Abadie, Diamond, and Hainmueller (2010) and chooses weights $\nu \in V, \omega \in W$ to **minimize MSPE in the pre-intervention period**.

By default it does this for the **full pre-intervention period** $t = 1, \ldots, T_0$:

$$\min_{V,W} MSPE_{t=1}^{T_0} = \frac{1}{T_0} \sum_{t=1}^{T_0} \left(Y_{1t} - \sum_{j=2}^{J+1} \omega_j(V) Y_{jt} \right)^2$$

Optimization Window

Specifying `optimization_window` as the range $1 \leq t_s, \ldots, t_e \leq T_0$ forces only the specified time window to be used when calculating MSPE:

$$\min_{V,W} MSPE_{t_s}^{t_e} = \frac{1}{t_e - t_s + 1} \sum_{t_s}^{t_e} \left(Y_{1t} - \sum_{j=2}^{J+1} \omega_j(V) Y_{jt} \right)^2$$

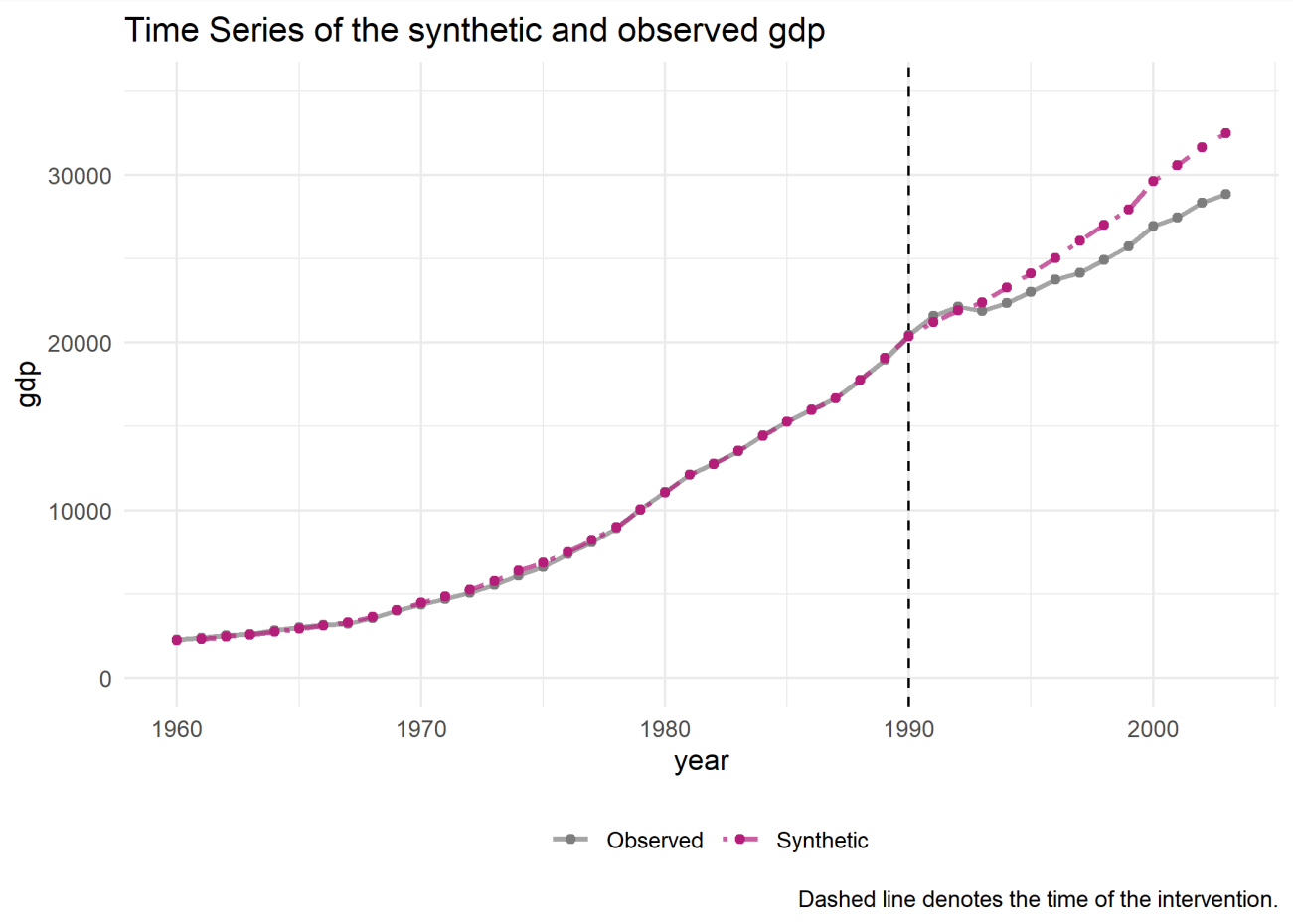
- In the background of the `synth_weights()` function, this is done by filtering the data to `filter(time %in% optimization_window)`

Optimization Window: Application

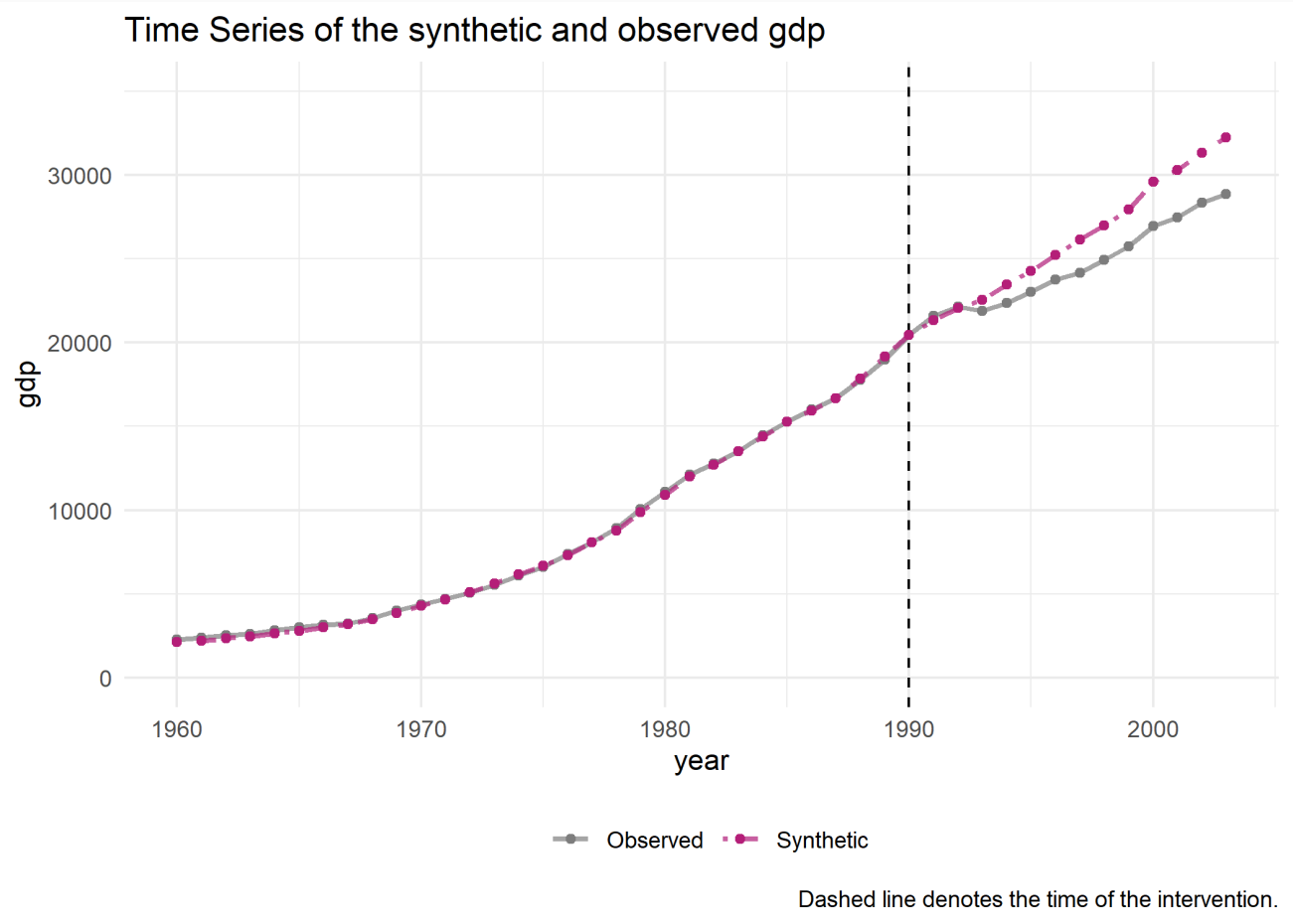
Abadie, Diamond, and Hainmueller (2010), which we're replicating, uses the period 1981-1990 as its optimization window

- Use 1971-1980 as the training period
- Use 1981-1990 as the validation period
- Use predictors in 1971-1980 to build V to minimize RMSPE in 1981-1990
- Use V to choose W to maximize balance in the decade right before reunification (1981-1990)

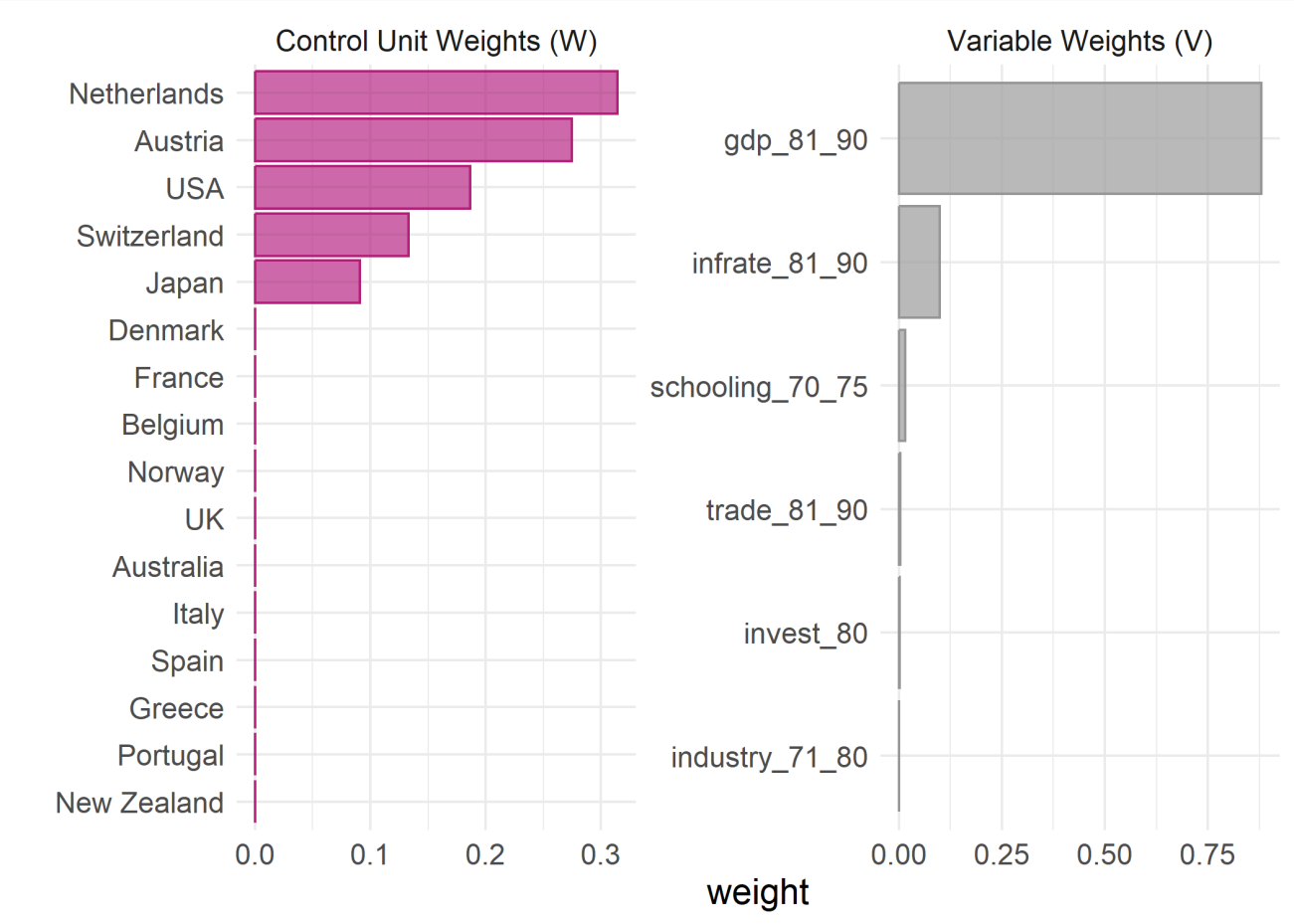
Comparing: 81-90 Window



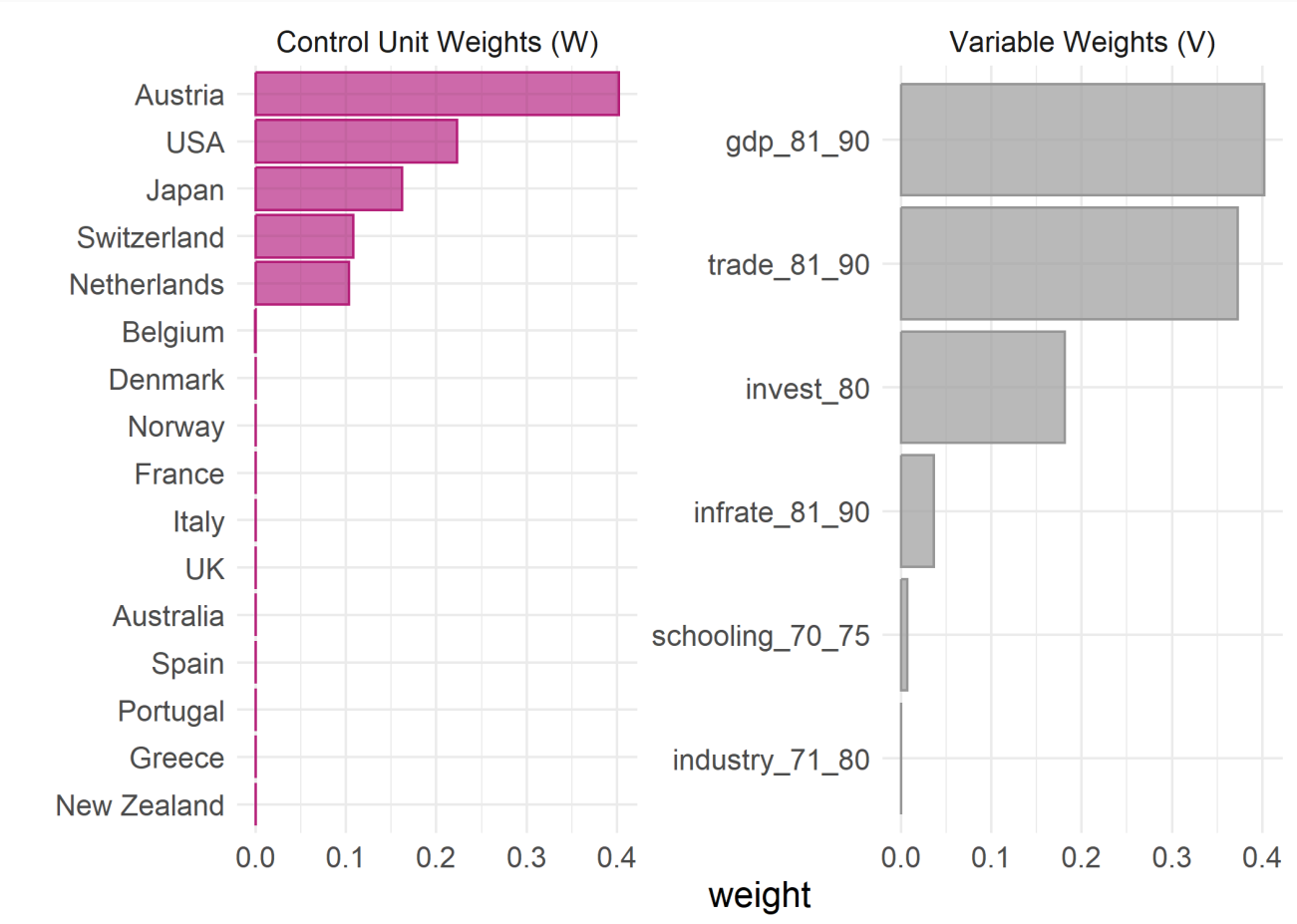
Comparing: 61-90 Window



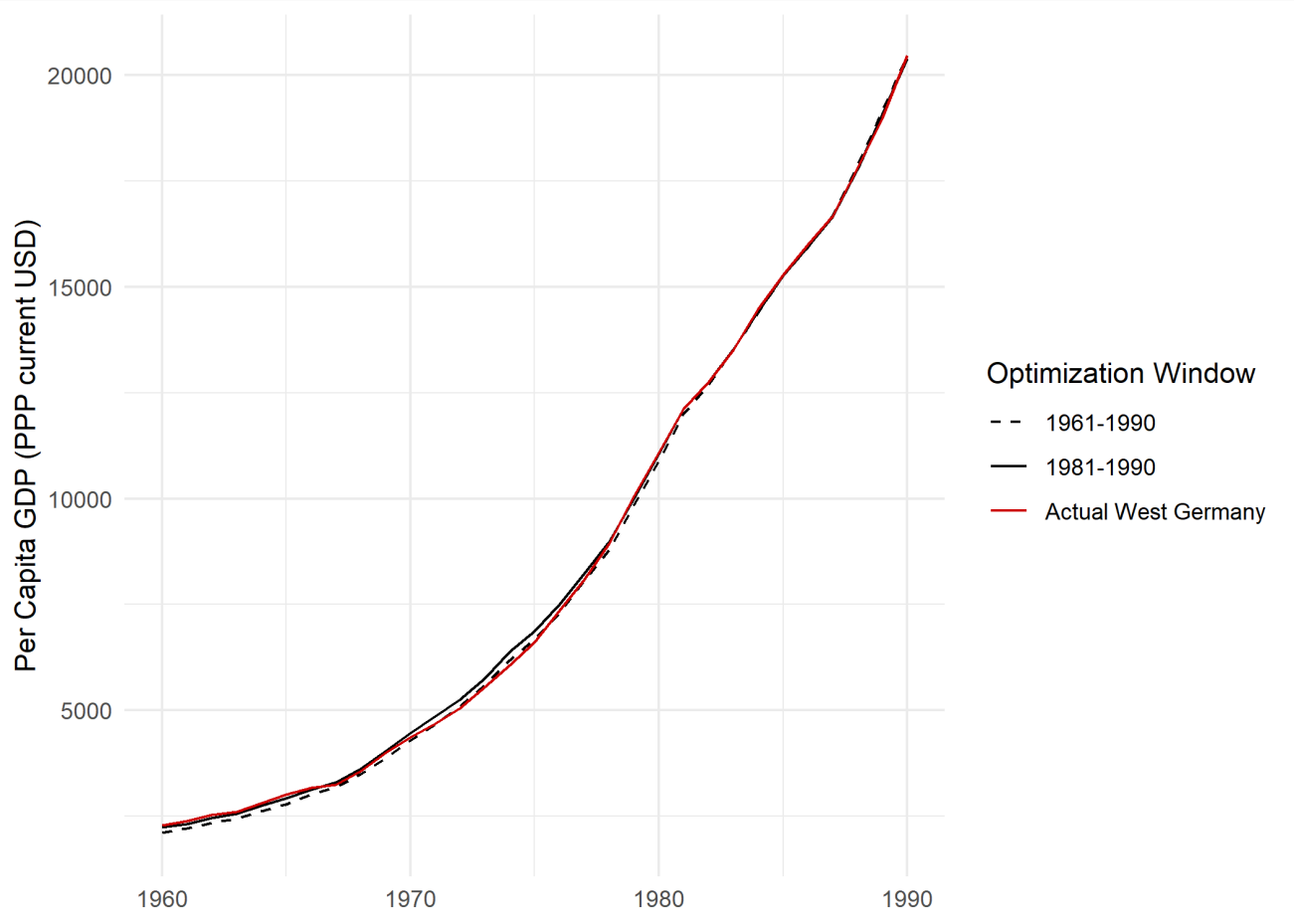
Comparing Weights: 81-90



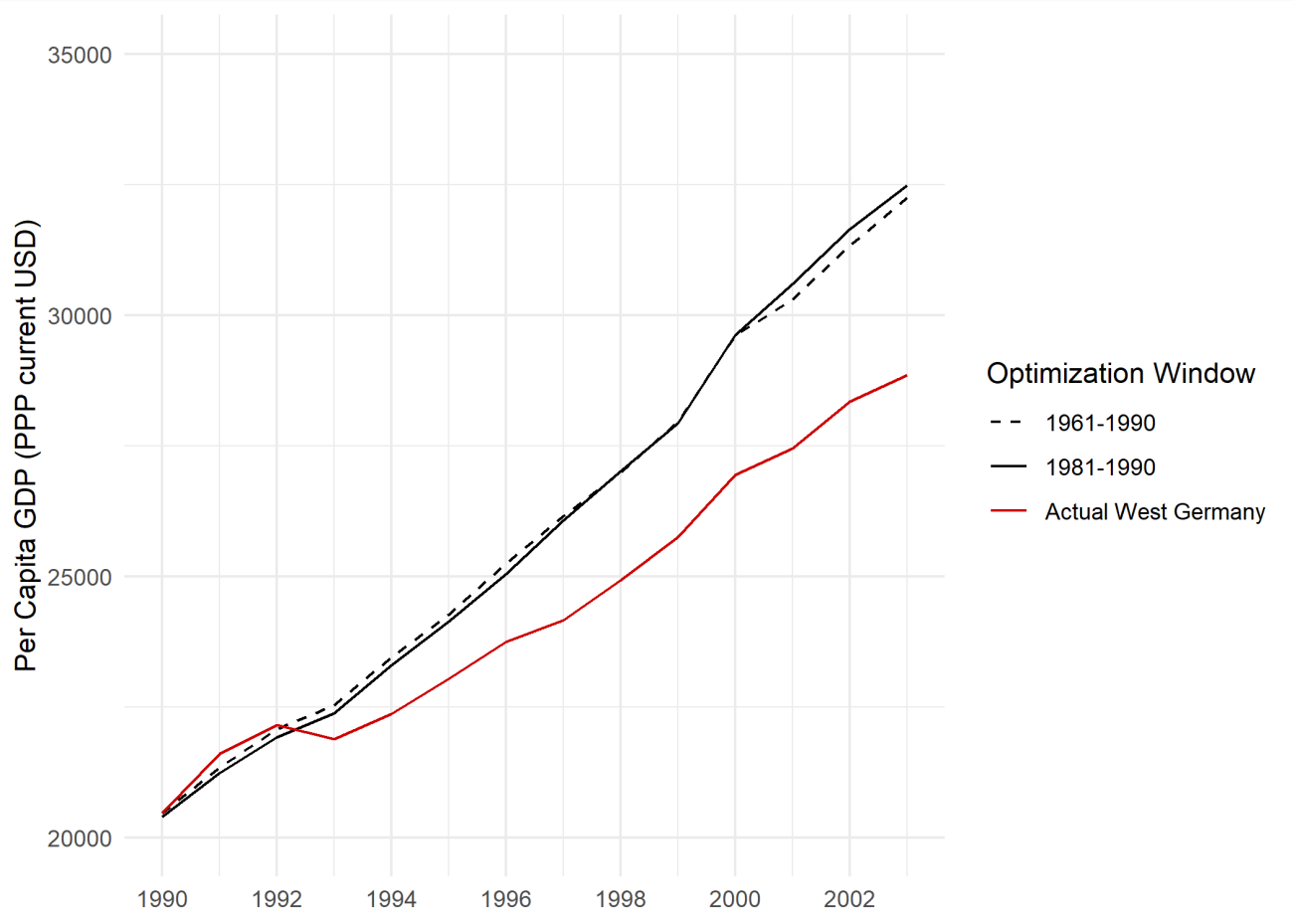
Comparing Weights: 61-90



Comparing: Pre-Period



Comparing: Post-Period



Punchline

- Lack of "uniqueness" of synthetic controls (e.g. "local minima")
  - Two different sets of unit/predictor weights
  - Similar pre-treatment fit, post-treatment path
  - Average post-treatment difference between the two SC: $7.97 (0.03% of dependent variable mean)

```
## # A tibble: 1 × 2
##   mean_gdp mean_diff
##      <dbl>     <dbl>
## 1   24709.     -7.97
```

Impact of Variable Scale

Impact of Variable Scale

What if we rescaled one of our variables instead?

Thinking back to our objective function with *just* Y and **no other predictors:**

$$\min_{V,W} MSPE_{t=1}^{T_0} = \frac{1}{T_0} \sum_{t=1}^{T_0} \left(Y_{1t} - \sum_{j=2}^{J+1} \omega_j(V) Y_{jt} \right)^2$$

A rescaled Y *shouldn't* change things, as everything is being scaled equally

Impact of Variable Scale

However, if we instead have a set of multiple predictors, then it **should affect things**!

- Re-scaling Y or X_k changes its proportional representation in the overall squared difference term
  - Ex: a squared difference of inflation rate as percent is 10000 times larger than as a proportion!

Impact of Variable Scale: Y

Let's test this first re-scaling Y in two ways in the **multiple predictor case**:

1. Re-scale our dependent variable in millions of dollars in GDP (instead of dollars)
2. Standardize our dependent variable

Re-scale GDP in Millions

```
deu ← mutate(deu,
             gdp_m = gdp/1000000,
             gdp_z = (gdp - mean(gdp, na.rm = T))/sd(gdp, na.rm = T))
```

Impact of Variable Scale

Generate predictors exactly as before:

```
synth_m ← synthetic_control(deu,
      outcome = gdp_m,
      unit = country,
      time = year,
      i_unit = "West Germany",
      i_time = 1990,
      generate_placebos = F
    ) %>%
  generate_predictor(time_window
= 1981:1990,
                     gdp_81_90 =
mean(gdp, na.rm = T),
                     trade_81_90
= mean(trade, na.rm = T),

infrate_81_90 = mean(infrate,
na.rm = T)
  ) %>%
```

```
synth_z ← synthetic_control(deu,
      outcome = gdp_z,
      unit = country,
      time = year,
      i_unit = "West Germany",
      i_time = 1990,
      generate_placebos = F
    ) %>%
  generate_predictor(time_window
= 1981:1990,
                     gdp_81_90 =
mean(gdp, na.rm = T),
                     trade_81_90
= mean(trade, na.rm = T),

infrate_81_90 = mean(infrate,
na.rm = T)
  ) %>%
```

Impact of Variable Scale

Estimate each model exactly as before:

```
synth_m ←
generate_weights(synth_m,

optimization_window = 1981:1990)
%>%
  generate_control()
```

```
synth_z ←
generate_weights(synth_z,

optimization_window = 1981:1990)
%>%
  generate_control()
```

Comparing: Unit Weights

Comparing the unit weights:

```
## # A tibble: 16 × 4
##    unit          Main `Y Rescaled` `Y Standardized`
##    <chr>        <dbl>        <dbl>             <dbl>
##  1 Netherlands  0.31          0.31              0.31
##  2 Austria      0.27          0.27              0.27
##  3 USA          0.19          0.19              0.19
##  4 Switzerland  0.13          0.13              0.13
##  5 Japan        0.09          0.09              0.09
##  6 Australia    0             0                 0
##  7 Belgium      0             0                 0
##  8 Denmark      0             0                 0
##  9 France       0             0                 0
## 10 Greece       0             0                 0
## 11 Italy        0             0                 0
## 12 New Zealand  0             0                 0
## 13 Norway       0             0                 0
## 14 Portugal     0             0                 0
## 15 Spain        0             0                 0
## 16 UK           0             0                 0
```

Comparing: Predictor Weights

Comparing the predictor weights:

```
## # A tibble: 6 × 4
##   variable          Main `Y Rescaled` `Y Standardized`
##   <chr>            <dbl>        <dbl>             <dbl>
## 1 gdp_81_90         0.88         0.88              0.88
## 2 infrate_81_90     0.1          0.1               0.1
## 3 schooling_70_75   0.02         0.02              0.02
## 4 trade_81_90       0            0                 0
## 5 industry_71_80    0            0                 0
## 6 invest_80         0            0                 0
```

Rescale X

... huh.

Okay... well What if we rescaled one of our X's instead? Say, inflation rate as a percent?

```
deu ← mutate(deu, inf_pct = infrate*100)
```

```
synth_x ← synthetic_control(deu,
      outcome = gdp,
      unit = country,
      time = year,
      i_unit = "West Germany",
      i_time = 1990,
      generate_placebos = F
    ) %>%
  generate_predictor(time_window = 1981:1990,
                     gdp_81_90 = mean(gdp, na.rm = T),
                     trade_81_90 = mean(trade, na.rm = T),
                     infrate_81_90 = mean(inf_pct, na.rm = T)
  ) %>%
  generate_predictor(time_window = 1971:1980,
```

Rescale X

```
## # A tibble: 6 × 4
##   variable          Main `Y Rescaled` `X Rescaled`
##   <chr>            <dbl>        <dbl>        <dbl>
## 1 gdp_81_90         0.88         0.88         0.88
## 2 infrate_81_90     0.1          0.1          0.1
## 3 schooling_70_75   0.02         0.02         0.02
## 4 trade_81_90       0            0            0
## 5 industry_71_80    0            0            0
## 6 invest_80         0            0            0
```

Rescale X

Wait, what?

It turns out that, while the standard SCM algorithm *isn't* X/Y scale-invariant, **tidysynth** methods *are*.

Looking into its **source code on Github** reveals why: it **internally standardizes all variables!**

Punchline

- Synthetic control **is invariant to units/linear scaling of Y** but **only when Y is the sole predictor**
- Canonical synthetic control **is not invariant to units/linear scaling of Y/X with multiple predictors**
  - Good idea to standardize your predictors *before* estimation
- **tidysynth** methods **internally standardize** all predictors and **are invariant to units/linear scaling of Y/X**
  - No need to pre-standardize!